Lecture 3 File Systems

2022年11月10日 6:11

Introduction

Every FS has a unique **layout** of the storage device, which defines what the things stored on the device are and where they are.

The set of FS **operations** defines how the OS should work with the FS layout. In other words, **the OS knows the FS layout** and works with that layout.

The process invokes **system calls**, which in turn invoke the FS operations, to access the storage device.

FS, OS, and storage device

An OS supports one or more FS. An FS can be accessed by more than one OS.

An FS must be stored on a device, but a device may or may not contain any FS.

A storage device is just a dummy container. It does not know what or how many file systems are stored on it. The OS instructs the storage device how to store the data.

What are we going to learn

- **Two basic things**: **files** and **directories**.
- The **layout** of the file systems. The layout may affect the speed of operations on the file systems, the reliability of using the file systems, and the allocation and deallocation of disk spaces.
- **FAT32** (Microsoft) and **ext2/ext3/ext4** (Linux) file systems.
- Interactions between the OS kernel and the processes. Specifically, why Linux can support so many file systems.

Files

Why we need files

Storing information in memory is good because **memory is fast**. However, memory **vanishes** after reboot. Files provide **long-term** information storage (they are persistent). Files can also be **shared objects** for processes to access concurrently.

Filenames

When a file is created, a **name** must be given (extremely important to users). There are naming conventions:

- Length of the name
  - DOS: 8 characters of name + 3 characters of extension.
  - Windows NT, Linux, and macOS: 255 characters of name and extension combined.
- Case sensitivity
  - Windows and newer macOS: case insensitive.
  - Linux and older macOS: case sensitive.
- File extension
  - Application dependent.
  - For *gcc*, the extension is important.
  - For a text editor, the extension is not important.
  - Windows takes the file extension very seriously.

Pathname vs. Filename

- The **pathname** is **unique** across the entire file system.
- The filename is not unique across the file system.
- The filename is unique within the directory that it resides in.

The OS kernel translates the pathname into **a set of data addresses** on the device. That means the pathname is the **key**! If the pathname were not unique, the OS would not be able to locate the data correctly.

What are going to be stored

- Filename.
- File content.
- File size: attribute of the file. Note that the file can contain null bytes, so that string operations cannot be used. Therefore, storing the file size is necessary for us to know where the file content terminates.

File attributes

File attributes are important to the file system. A file can have empty content, but not an empty set of attributes.

- Important: size, owner, permission.
- Less important: creation time, modification time, last accessed time.
- Dynamic attributes (appear when the kernel is manipulating the files): file pointer, open file count, …

File attributes are **FS-dependent**, not OS-dependent.

```
$ ls -l final_exam_solution.txt
-rw-r—r—. 1 yt2475 yt2475grp 4096 Oct  7 07:35 final_exam_solution.txt
```

	File attribute	FAT32	NTFS	ext2/3/4
1	Permission	✗	✓	✓
2	Link count	✗	✗	✓
3	User ID	✗	✓	✓
4	Group ID	✗	✓	✓
5	File size	✓	✓	✓
6	Modification time	✓	✓	✓
7	File name	✓	✓	✓

Of course, the column for ext4 should be all ✓, because I ran this example on Linux. 😜

Using the command *stat*, it calls the *stat()* system call, which can obtain a *struct stat* (note that it contains an *off_t st_size* which gives the total size of a file in bytes).

Two examples using *stat()* system call:

- Check whether a file **exists** or not without opening that file:

```
int main(int argc, char **argv) {
  struct stat file_stat;
  if (stat(argv[1], &file_stat) == -1)
    exit(1);

  printf("file size of argv[1] = %d\n", file_stat.st_size);
}
```

- Check whether a filename points to a file or a directory:

```
int main(int argc, char **argv) {
  struct stat file_stat;
  if (stat(argv[1], &file_stat) == -1)
    exit(1);

  if (S_ISDIR(file_stat.st_mode))
    printf("%s is a directory\n", argv[1]);
  if (S_ISREG(file_stat.st_mode))
    printf("%s is a regular file\n", argv[1]);
}
```

Summary

Every file has its **unique pathname**. Its pathname leads you to its **attributes** and the **file content**. These are the two important components of a file, and they are usually stored in separate locations.

File types

- Regular file
  - Text files
  - Binary files
- Directory file
  - Will be discussed in the Directory section.
- Link file
  - A **link** is an **alias** of a file. In Windows, we have the **shortcut**. In Unix/Linux and macOS, we have the **hard link** and **symbolic link**. Both the shortcut (just an ordinary file for Windows) and the symbolic link (a special type of file for Linux) share the same concept. A link file is created and stores the **pathname** of another file. To create a symbolic link, we use *ln -s filename linkname*.
  - For example, in the Lab 3 autograder, *7.in -> 6.in* is a link file, only 4B, which is created using *ln -s 6.in 7.in*
- Device file
  - In Unix/Linux, most devices are represented as files. For example, */dev/had* is the first IDE hard disk drive, */dev/sda* is the first SATA/SCSI hard disk drive, */dev/fd0* is the first floppy drive, etc. Don't try anything like *cat something > /dev/sda*, since you will overwrite and erase all the files on your disk!
  - Unix/Linux creates some **pseudo-devices** for convenience.
    - */dev/zero*: outputs an constant (endless) stream of zeros.
    - */dev/urandom*: outputs a constant (endless) stream of random bytes.
    - */dev/null*: a black hole, so whatever data written to this device will be discarded.

Regular files

Text files: contain text only. They are human-readable, and can be inspected using *cat*. Examples include C programs, HTML documents, JSON files, etc.
Binary files: contain any data. They are not human-readable, and can be inspected using *xxd*. Examples include executable files, images, videos, encoded data, etc.

The command *file* can distinguish the type of a file. When distinguishing binary files, *file* command checks a special pattern called the **magic number**, which is stored at the beginning of most binary files. For example, GIF files always start with the string "GIF87a" or "GIF89a", PDF files always start with "%PDF-", etc.

The command *file* knows all the magic numbers of all file types. They are listed in the file */usr/share/file/magic*. However, *file* command does **not** check the extension of the filename. This means, if we manually modify the first few characters of a file to be a different magic number, *file* command will misread the file type regardless of the real file extension.

The anatomy of opening a file

Step 1: The process sends a pathname to the OS.
Step 2: The OS looks for the **file attributes** of the target file on the disk.
Step 3: The disk returns the file attributes.
Step 4: The OS then associates the attributes to a number, which is called the **file descriptor**.
Step 5: The OS returns the file descriptor to the process.

Opening a file only involves the **pathname** and the **attributes of the file**, instead of the file content!

The anatomy of reading a file

Step 1: The process sends a file descriptor to the OS.
Step 2: The OS reads the file attributes and uses the stored attributes (data location, current-file-position pointer, etc.) to **locate the required data**.
Step 3: The disk returns the required data.
Step 4: The OS keeps the data in a **buffer cache** and copies the data to the process. (The first time we read a file, we need to fetch the file from the disk, but subsequent read operations can just serve the file contents from the buffer cache.)

Sequential access vs. random access

Sequential access: only found in old systems. Bytes or records must be read in order, starting from the beginning. There is no *seek* operation (though *rewind* 倒带 is usually allowed). It is just like a cassette tape.
Random access: applications can read bytes at **any position**, in **any order**. *Seek* operation is supported.

Some system calls

System call	Description
open	Open a file. Create a set of dynamic attributes inside the kernel. This system call may create a new file, and it involves the update in the directory entries.
close	Flush all the outstanding data to the storage media. Clear the dynamic attributes and have the static file attributes updated.
read	Read data from the opened file. The file pointer will be updated after a successful read.
write	Write data to the opened file. The file pointer will be updated after a successful write.
lseek	Change the position of the file pointer.
link	Create a hard link.
symlink	Create a symbolic link.
stat	Read the attributes of a file.
rename	Change the pathname of a file.
unlink	Remove an existing file.

Directories

What is a directory

A directory is also a **file**. Hence, a directory has file content, and for most FS, it also has file attributes. A directory is **an array of directory entries**, recording all the files and directories that belongs to it. In a directory entry, there are names of the stored files, as well as their corresponding attributes (or the locations of their corresponding attributes).

Locating a file using the pathname

Step 1: The process sends the unique pathname "/bin/ls" to the OS.
Step 2: The OS retrieves the directory file of the root directory "/" from the disk.
Step 3: The disk returns the directory file of "/".
Step 4: The OS looks for the name "bin" in the directory file.
Step 5: If found, then the OS retrieves the directory file of "/bin" **using the information of the file attributes of "/bin"** from the disk.
Step 6: The disk returns the directory file of "/bin".
Step 7: The OS looks for the name "ls" in the directory file of "/bin".
Step 8: If found, then the OS knows that the file "/bin/ls" is found, and it starts the previously-discussed procedure to open the file "/bin/ls".

How to read a directory

It is very similar to the open-read-close pattern. Along with the *stat()* system call, you can implement a complete */bin/ls* program.

```
DIR *dir;
struct dirent *entry;
dir = opendir("/");  // open a directory
while ((entry = readdir(dir)) != NULL) {  // read a directory entry until there is no further entries
     printf("%s\n", entry -> d_name);
}
closedir(dir);  // close the opened directory
```

A **directory traversal process** is invoked whenever you open a file, execute a program, or list the content of a directory (etc.)

File creation

File creation is just **updating the directory file**!

Some system calls

System call	Description
mkdir	Create a new, empty directory.
rmdir	Remove an empty directory.
open	Actually, the open() system call is also used to open a directory. Library function: opendir().
close	Actually, the close() system call is also used to close a directory. Library function: closedir().
getdents	Read the directory entry. Library function: readdir().

File system design

What is a file system

A **file system** is about how the OS stores and locates a **file**, and how the OS stores and locates a **directory**. Also, it is about how the OS manages the storage in an **efficient** (about the speed and the storage utilization) and **reliable** (about whether we can get back what have been saved) way.

- Application level (user)
  - What are the **entities** and the **services** that the FS is providing to the applications.
- Logical representation level (user, kernel)
  - How does the FS **represent the entities in a logical way**.
- File organization level (kernel)
  - How does the FS **organize the entities on the storage device**.
- Device control level (kernel)
  - How does the FS **control the devices to read and write the entities**.

Logical representation level

In a file system, the logical representation goes **partially** to the users and **mostly** to the kernel.

- The users have to know what entities and services the FS provides, logically.
- The users do not need to know how the logical entities are implemented.

This is an FS-specific issue, which will be discussed later in detail through the FS of Linux.

File organization

Attempt #1: easy peasy lemon squeezy: just write files sequentially (scan from byte 0, and start writing a new file whenever we find the next empty space on the disk).

- It is **extremely slow** in retrieving data and writing data, since we will need to search through the whole disk space.
- When deleting files, there are holes left on the disk. Then writing a file in that hole may **overwrite** other files.

Attempt #2: keep a table of the starting and ending addresses of all files. Let's call it the file system table.

- I can quickly search any files.
- I can quickly find the next empty space.
- The file system table shows the holes on the disk, so the file overwriting problem can be avoided.
- The free space of the disk is **not used wisely**. A large file may not find a suitable hole, even if there is actually enough space. We call this issue the **external fragmentation** problem. The allocated space is fragmented into chunks, and there are many holes in between.

Attempt #2': align all files in a sequential way again. This means, the allocated space stays at the front of the disk, and the empty space stays at the end of the disk. After such operations, we also correspondingly rewrite the file system table.

- We call this procedure **defragmentation**.
- Defragmentation is very expensive. This procedure involve all the files on the disk, thus very slow. Moreover, the disk becomes **unavailable** during the defragmentation.
- If a file wants to **grow**, but there is no empty space adjacent to it, then the solution to this problem is very expensive.

This file system implementation is called **contiguous** 连续的 **allocation**. It is the best choice when <u>no files will be deleted from the disk</u>, and <u>all file sizes are fixed</u>. Real-life example include: ISO 9660, Joliet (file systems for CD/DVDs). CD/DVDs are read only and will not be modified.

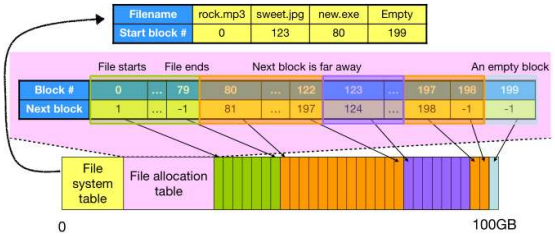
Attempt #3: fille the new file in the empty space **block by block**. That is, divide the disk space into small and equal-sized blocks, and divide the file in the same manner as well. In this way, the new file can be filled into the available empty spaces effectively. But now each file will have multiple starting and ending addresses.

Attempt #3.1: all files are organized as a **linked list** of data blocks. The **free space** is also organized as a linked list.

- It is easy to implement. The pointer value is just the ID (or the address) of the next block.
- The file system table is **smaller** than contiguous allocation.
- No external fragmentation problem.
- Files are allowed to **grow** and **shrink**.
- Free space is well-organized.
- **Random access problem**: it is expensive to access a (random) specific block. In the worst case, if you want to read the last block of your file, you have to traverse from the first block on the disk all the way until you reach the last block. This involves a large number of disk accesses.
- **Internal fragmentation problem**: a file is not always a multiple of the block size. Hence, the last block **may not be fully filled**. This remaining space will be wasted because no other files can be allowed to fill such space.
- **Trade-offs** in choosing the block size:.
  - If the block size is small, the total number of blocks is large. The number of **disk accesses** increases as each file contains more blocks. The list representation itself will also take more space, so there is **less space for real content**.
  - If the block size is large, the internal fragmentation problem will be **worsened**.

Attempt #3.2: store the linked list as a table. We call this approach the **file allocation table (FAT)** approach.

- All the information about the next block are **centralized** as a table.
- The entries in the table are stored **contiguously as an array**.
- Each entry corresponds to **one file allocation block** in the storage device.



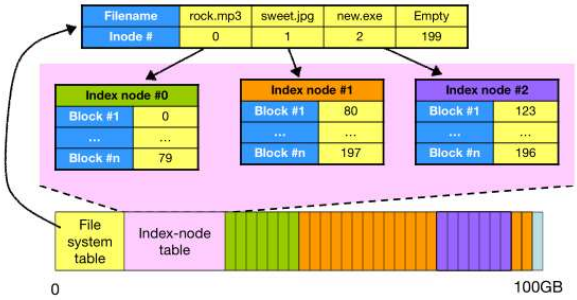
- The size of the FAT depends on the size of the storage device and the block size (their quotient is the number of the entry, then multiply the number of bytes for each entry).
- The file system table and the file allocation table can be **cached in memory** inside the kernel for fast block lookup.
- **No random access problem** because the block lookup is done in memory.
- The **file allocation table** takes a lot of space (but the benefit outweighs this drawback).
- FAT12, FAT16, FAT32 (Microsoft), USB drives, SD cards. NTFS does a similar thing, but more complex (widely used by newer Windows system).

If FAT, a block is called a **cluster**. Different versions of FAT support different number of clusters. For instance, FAT16 means 16 bits for the cluster address length, so there can be 2^16 clusters (note that FAT32 uses 28 bits but not 32 bits). Different versions of FAT also support **different maximum file sizes**. Let *L* be the length of the cluster's address, and let *S* be the size of a cluster, then the maximum file size is *2^L * S*.

See also [FAT32 in action].

The FS has to load the entire file allocation table into memory in order to know the locations of allocated data blocks. The file allocation table itself may contain thousands of blocks, which takes a lot of memory. However, we may not need to store the entire set of block location information. If we can break the table into smaller pieces, then the FS only needs to read a few blocks in order to know the locations of all allocated data blocks.

Attempt #4: index node (inode) approach. We can divide the file allocation table based on files, so as to form an index-node table. It contains various index nodes (each file is assigned to an index node), and each index node contains the block locations for that specific file.



- The file system table maintains the mapping from the filenames to the index nodes (inodes).
- An inode stores the addresses of all the data blocks.
- All the inodes are stored in the inode table.
- The size of the inode table determines the number of files that the FS can store.
- A read operation on the **file system table** and a read operation on the **inode table** together locate all the data blocks of a file, and that **consumes less memory** than FAT.
- The size of an inode has to be **variable**, since the length of the list of block addresses is variable. Thus is becomes **hard for the FS to read the inode table efficiently**.

Attempt #4.1: a new idea to index a file's block allocation. We introduce indirect blocks. While direct blocks store data, indirect data blocks store addresses of other blocks. There can be multiple layer of indirect blocks, meaning that indirect blocks can also store addresses of other indirect blocks.

- Each inode has a **fixed size**, while it increases the **supported file size** significantly (exponential to the number of layers).
- At the beginning, the addresses of the data block are stored in the **direct block addresses** of the inode. After the direct block addresses have been used up, the **indirect block addresses** will be used.
- For instance, if there are 12 direct blocks, 1 indirect block, 1 double indirect block, and 1 triple indirect block. Suppose the block size is 2^x bytes and the address length is 4 bytes. Then max file size would be (12+2^x/4+(2^x/4)^2+(2^x/4)^3)2^x=12 2^x+2^(2x-2)+2^(3x-4)+2^(4x-6). That gives, if a block is 4KB, then the maximum file size supported would be 4TB, which is good enough.

See also Ext2/3/4 in action.

Free-space management

We would need to handle the free space on a disk when we delete a file, create a file, or initialize a file system. The goal is to keep track of **every free block** in the file system, and it should be as **FS-independent** as possible.

Attempt #1: allocate data blocks to store the list of addresses of the free blocks. The FS needs to remember the starting block of the list of free blocks (could be hard-coded).

- **Too much space** to maintain such a list of free blocks. For instance on a 128GB hard disk drive, assume that each block has 1KB, then there would be 2^27 blocks, and if a block address has 4 bytes, the list would be 512MB (far too large).

Attempt #2: bitmap. What we need to represent is a series of boolean values (a bit of 0 represents and unallocated block, while a bit of 1 represents an allocated block). Thus, we can construct a **bitmap of block allocation**.

- ext2/3/4 uses this implementation.
- Same assumption as above, there would be 2^27 blocks, and since each block only takes 1 bit, the space needed to maintain a bitmap of allocations is just 16MB.

FAT: In FAT16, there is almost no free space management. The FAT only indicates which clusters are free. In FAT32, there are fields that manage the address of the first free cluster, and the total number of free clusters.

- This needs only little space.
- However, it is slow to calculate the total number of free space (in FAT16).
- Also, it is slow to allocate free clusters for new data. A linear search must be performed from the start of the FAT (in FAT16) or from the start of the first free block (in FAT32).

Supporting containers (*i.e.*, directories)

A **directory** is stored as a set of structured data. From an FS's point of view, a directory is just **a list of directory entries**. Therefore, a directory is considered as a special type of file.

- A directory usually takes only one data block. Therefore, there is a limit on the number of directory entries in each directory.
- A directory entry must at least contain the filename and the information about the data blocks of the file.

The file system should know where the **root directory** is in order to know how to traverse the directory tree.

Storing file attributes

This is FS-dependent. In FAT16 and FAT32, file attributes are stored inside a directory entry. In ext2, ext3, and ext4, file attributes are stored inside an inode.

File system information

We cannot hardcode information such as the size of a block, the number of allocated blocks, the number of free blocks, the location of the root directory, etc. This is because different storage devices have different needs. Therefore, we have to read these information from somewhere in the disk.

FAT, NTFS: the **boot sector** is the **first 512 bytes** of the FS to store all the FS-specific data. For instance, bytes per sector, sectors per cluster, cluster number of the root directory, number of FATs, number of sectors of the reserved area, etc.

ext2/3/4: the **superblock** is a 1024-byte region to store all the FS-specific data. For instance, number of inodes, number of blocks, number of unallocated inodes, number of unallocated blocks, block size, size of each inode structure, etc. It also stores the current mount count and the maximum mount count (a file system check *fsck* will be performed if the maximum mount count is reached).

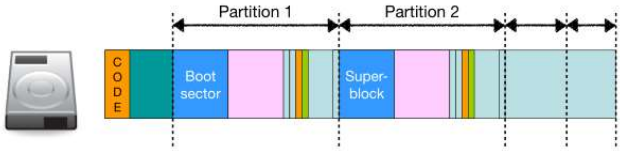
Disk partitions

A **disk partition** is a logical space to host a file system. There may be several file systems contained in the disk, so there may be several disk partitions.

Before the disk partitions, there is an **MBR (Master Boot Record) code** in the very beginning, which specifies which partition to boot, an following the MBR code is a **partition table** storing the first sector and the length and type of a partition.

Why do we need to partition a disk?
- A smaller file system is **more efficient**.
- **Multi-booting**: You can have Linux + macOS + Windows installed on a single disk.
- You can have one logical drive to store the OS-related files, one logical drive to store personal documents, and another logical drive to store movies. Therefore, there would be better data organization.

Formatting a disk means creating and initializing a file system.
- DOS/Windows: format.exe
- Linux: mkfs.fat, mkfs.ext2, mkfs.ext3, mkfs.ext4, …
- macOS: diskutil



FAT32 in action

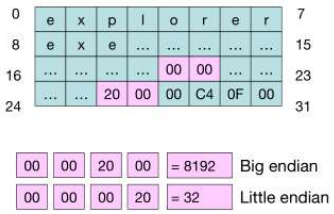
- Cluster size: 512B, 1KB, …, 128KB, 256KB.
- Cluster address length: 28 bits.
- Maximum file size: (4GB - 1) because the file size filed is 4-byte long (0xFFFFFFFF = 16^8).
- Maximum partition size: cluster size * 2^10 * 2^28 bytes. (For example, 32 KB cluster size has maximum partition size 8TB.)

Boot sector
- Beginning of the file system, contain all information about the FS.
- Use *mkfs.fat* to format a disk, and use *fsck.fat* to verify that disk.
- How many bytes per logical sector, how many bytes per cluster -> the size of one block
- How many reserved sectors -> the number of sectors reserved at the very beginning of the FS
- Which byte/sector that the first FAT start at (there may be multiple FATs to survive possible damage of some FAT)
- **Root directory start at cluster 2 (arbitrary size)** -> this is just Microsoft convention, there are no cluster 0 or 1
- Which byte/sector that the data area starts at

Reading a directory
- Step 1: locate the **root directory**. Read the boot sector.
- Step 2: find the **directory entry**. Single dot is the current directory, double dot is the parent directory (parent of root is still cluster 2). Each directory entry has filename, attributes, and a cluster number (which is the cluster number of the first cluster of that file).
- Step 3: follow some cluster, read and return all directory entries.
- Step 4: read and return the entire directory entry structure (a 32-byte structure, Bill Gates did not consider permission and user information, so there are just those simple attributes).

Bytes	Value
0-0	1st character of the filename (0x00 or 0xe5 means unallocated).
1-10	7+3 characters of filename + extension.
11-11	File attributes (e.g., read only, hidden).
12-12	Reserved.
13-19	Creation and access time information.
20-21	High 2 bytes of the first cluster address (0 for FAT16 and FAT12).
22-25	Written time information.
26-27	Low 2 bytes of first cluster address.
28-31	File size.

  - Note that the cluster address are stored in two separate locations, since back in FAT16 and FAT32, they needed only 2 bytes to store the first cluster address. For compatibility, since the last 4 bytes are reserved for the file size and the 4 bytes after the high 2 bytes of the cluster address are reserved for written time information, so the low 2 bytes had to be stored separate from the high 2 bytes.
  - **Big endian** vs **little endian**: the former stores small value in larger addresses (everyday life, SPARC, network protocols, etc.), and the latter stores small value in smaller addresses (Intel, AMD, etc.) **Endianness** is about byte ordering, which is a computer architecture issue. Therefore, suppose we have 20-21 bytes as **00 00**, and 26-27 bytes as **20 00**, in big endian, it is organized as 00 00 20 00 = 8192, but on Intel machines (Bill Gates uses back them), in little endian, they should be instead organized as 00 00 00 20 = 32, as illustrated below:



  - **LFN (long filename)** directory entries: in the old days, Bill Gates set the rule that every file should follow the **8.3 naming scheme**. Yet he removed such a constraint in FAT32 by a backward-compatible but duper ugly design.

Bytes	Value (LFN directory entry)
0-0	Sequence Number.
1-10	File name characters (5 characters in Unicode).
11-11	File attributes — always 0x0F for LFN.
12-12	Reserved.
13-13	Checksum.
14-25	File name characters (6 characters in Unicode).
26-27	Reserved.
28-31	File name characters (2 characters in Unicode).

Reading a file
- Step 1: read the first cluster (data block) from the directory entry.
- Step 2: look up the next cluster in the FAT and read that cluster.
- Step 3: repeat the process until an EOF entry is found in the FAT. Note that EOF > 0x0fffffff8.

Note that the way to read a file may not be sequential. The kernel usually needs to know all the cluster addresses before the actual reading starts. Besides EOF, unallocated (0), and allocated states, a cluster can also be **damaged**, *i.e.*, having bad sectors.

Writing a file
- Step 1: for appending, locate the cluster of the end of the file (just start from the first cluster and follow the FAT until you reach some cluster whose next is EOF).
- Step 2: look up the FSINFO structure to find the next free cluster. It is stored in the reserved space in the beginning of the disk, in the boot sector. It contains the number of free clusters, and the index of the next free cluster.
- Step 3: allocate new cluster by changing the FATs and FSINFO. Note that all/both FATs need to be updated if there are multiple ones of them.

Note: FSINFO structure is useful for finding the next one structure, but if more than one extra cluster is needed, a circular, next-available linear search is performed to find the next unallocated cluster. Nevertheless, after the writing finishes, the kernel still needs to search for one more unallocated cluster, since we need to update FSINFO. Therefore, in modern FS, no one really cares about FSINFO structure any longer.

Deleting a file
- Step 1: find the directory entry and the locations of the clusters in the FATs.
- Step 2: set all the next address fields in the FATs of that file to 0, which means unallocated.
- Step 3: update the FSINFO structure. (As previously mentioned, this is now rarely used. It's just something Bill Gates liked.)
- Step 4: change the first character of the filename to **0xE5**.

However, notice that the file is **not really deleted!** If we perform a search in all the free space, then, we will find all deleted file contents. Those data will persist until the deallocated clusters are **reused** (overwritten). This is a trade-off between performance (during deletion) and security (since back then, Bill Gates did not care about permission and security issues).

How to delete a file securely? See Remembrance of Data Passed: A Study of Disk Sanitization Practices.
How to recover a deleted file? Suppose that the target clusters have not been overwritten. If the file size is no more than one cluster, then the recovery can be easily done, since the first cluster address is still reachable. Note that files with the same suffix may also be found (for example, Yang and Tang after deletion are the same). If the file size is larger than one cluster, other clusters' addresses are all gone. However, because of the next-available search, clusters of a file are **likely to be contiguously allocated**. If not, you had better have the **exact file size** the and the **checksum** of the deleted file beforehand so that you can use a **brute-force** method to recover the file.

Summary

FAT is a simple yet pretty good file system.
- Space efficiency: 4 bytes overhead (FAT entry) per data cluster.

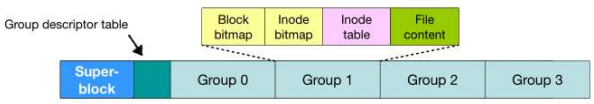
Security is not a consideration in the design.
- It does not maintain owners or permissions in file attributes.
- It employs **lazy deletion**.

It was widely deployed, and it still is.
- FAT is still dominant in SD cards and USB flash drives due to its high compatibility.

Ext2/3/4 in action

- The primary file system on Linux is ext4 (fourth extended file system).
- The default **block size** is 4KB (configurable from 1KB to 64KB).
- The default **block address** is 4 bytes, but can be configured to be 8 bytes.

Block groups
- The file system is partitioned into **block groups**. Each block group has the same internal structure. The **group descriptor table (GDT)** stores important information including:
  - The start of the block bitmap
  - The start of the inode bitmap
  - The start of the inode table



- The block groups are used for performance and reliability. First, it keeps the metadata and file contents **close together**. Thus, the disk head does not need to travel a long distance. Second, the metadata is scattered, so there is **no single point of failure**.

Superblock
- File system information (inode count, block count, first block, etc.)
- The layout of the FS (blocks per group, inodes per group, etc.)
- The inode usage information (first inode, inode size, etc.)

The metadata structure

- **Block bitmap** stores the block allocation status of the same block group.
- **Inode bitmap** stores the inode allocation status of the same block group.
- **Inode table** stores the contents of all the inodes of the same block group. Each inode has a fixed size specified in the superblock, and the inodes are stored sequentially (as an array).

Inode structure

Bytes	Value
0-1	File type and permission
2-3	User ID
4-7	Lower 32 bits of file sizes in bytes
8-23	Time information
24-25	Group ID
26-27	Link count
...	...
40-87	12 direct data block pointers
88-91	Single indirect block pointer
92-95	Double indirect block pointer
96-99	Triple Indirect block pointer
...	...
108-111	Upper 32 bits of file sizes in bytes

- Note that in the inode table, the index starts from 1 (not 0, and not 2 in FAT). There is no Inode #0.
- The first 10 inodes are **reserved** for special purposes.
    - Inode #1 always points to the bad blocks, Inode #2 always points to the root directory, and Inode #8 usually points to the journal data blocks.

Directory entry

- A directory is stored in the data blocks. A directory is made up of at least two directory entries: . (dot) and .. (dotdot).
    - Inode number (4 bytes)
    - Entry length (2 bytes) (this is 4 + 2 + 2 + filename length including the `\0` character, and remember the byte alignment that rounds this size to some multiple of 4)
    - Name length (2 bytes)
    - File name (up to 255 bytes)
- Note that the directory entries in ext2/3/4 have variable length, depending on the filename lengths.

Link file

- A **hard link** is a directory entry pointing to an existing file, and **no** new file content is created. The command is *ln*.
    - Conceptually, this creates a file with **two filenames**. Deleting only one of the directory entries will not delete the file content.
    - The **link count** field in the inode keeps track of how many directory entries are pointing to that file. When the link count becomes 0, the file content is removed, including the data blocks and the inode.
        - There is not delete system call in Linux, but we use the *unlink()* system call to decrement the link count.
    - Special hard links
        - The directory . (dot) is a hard link to itself.
        - The directory .. (dotdot) is a hard link to its parent directory.
    - When a regular file is created, the link count is always 1. When a directory is created, the link count is always 2 (. of itself and and an entry in its parent directory).
- A **symbolic link** is a new file. Unlike a hard link, a **new inode** is created for a symbolic link. The command is *ln -s*.
    - Where is the **target path** stored?
        - If the path is fewer than 60 characters, it is stored in the 12 direct block and the 3 indirect block pointers, since (12+3)*4=60. There is **no** need to allocate a new data block in this case.
        - If the path is more than 60 characters, one data block is allocated to store the path.

Writing a file

- Step 1: use a linear search in the inode bitmap to find an unallocated inode for the new file.
- Step 2: use a linear search in the block bitmap to find unallocated data blocks for the new file.
- Step 3: read the inode of the root directory, *i.e.*, Inode #2.
- Step 4: following the data block pointers, read the directory entry structure we want (repeat this step until we are going to construct the file).
- Step 5: read the inode for the directory that the file to be created is in, and the corresponding data blocks. Construct the directory entries.
- Step 6: Add a new directory entry with a the first free inode to that directory.

Deleting a file

- Step 1: read the inode and the data blocks following the path, and locate the inode of the file to be deleted.
- Step 2: read that inode and decrement the link count of the file to be deleted. Assume that now the link count has become zero.
- Step 3: deallocate the data block and the inode by setting the corresponding bits in the block bitmap and the inode bitmap.
- Step 4: change the entry length of the **previous** directory entry of the deleted file to the **sum** of its original length and the length of the deleted file. By doing this, when we traverse through the parent directory, we will technically skip the deleted file.

The change in the entry length aims to **skip the deleted entry**. When a new file is created in that directory, the FS should check if there is a hole that is big enough to store the new directory entry. And again, as in FAT32, the file is **not** really deleted. In ext2, the data block pointers will not be removed, but in ext3/4, the data block pointers will be removed.

Linux file system internals

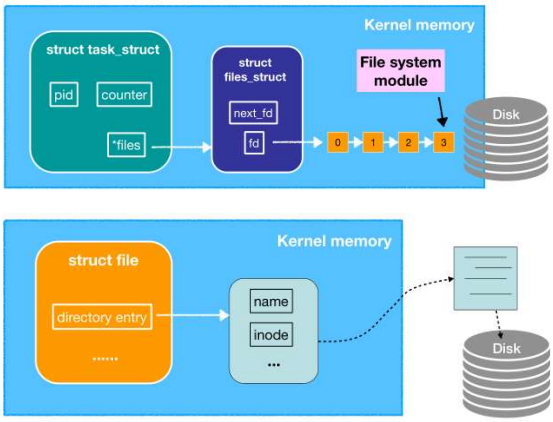
Kernel-process relationship

There is a strong relationship between a process and the opened files. The user program is given a **file descriptor** returned from the *open()* system call as an **abstract representation** of an opened file - it's just a number.

A process has its own set of opened files. It is managed inside the task structure *task_struct*. All the opened files are stored inside a structure of type *struct files_struct*.

By default, evert process initially has three opened files (0 - stdin, 1 - stdout, 2 - stderr). When the process calls the *open()* system call to open a file, the **file system module** reads the **metadata** of the file from the disk. The opened files will be represented as a structure and added to the process. A new file descriptor will be allocated for the new file.

The structure created contains the information about the **directory entry**. In turn, the directory entry contains the **name** and **inode** of that file. Note that the scope of a file descriptor is restricted inside the process. Different processes may use different file descriptors to the same file.



File system module

The **file system module** is the core of Linux file system support. It implements how Linux deals with the disk, and it implements the **virtual file system (VFS)**.

Virtual file systems

For each opened file, a set of **file system functions** is associated with it. They are just function pointers and are called **VFS functions**. The VFS functions are invoked by the FS-related system calls, and points to FS-specific functions (and that's why Linux can support many kinds of file systems, since file pointers can point to different functions specific to different file systems). The value of the function pointers are initialized when the file is opened, and determined by the location of the opened file (for instance, if the opened file is on an ext4 partition, the functions are mapped to the set of ext4 functions).

In short, same set of interface (VFS functions *e.g. open()*), but can invoke different sets of functions (which are FS specific).

- VFS allows the OS to support **multiple file systems** at the same time.
- It is easy to support a new file system in the future.
- However, VFS cannot utilize **special functions** of only some specific file systems, since otherwise the interface may fail for other file systems.

File system performance issues

Data fragmentation

The data fragmentation problem is about the **allocated blocks** of a file. A problem arises when they are **randomly scattered** on a disk.

Some basics about the hard disk drive:
- A hard disk drive has a movable **disk arm**, which has a **disk head** for reading and writing data. Data are stored sector-by-sector on the **platters** using magnetism. The platters are **spinning**, and therefore a spinning of the platters can retrieve a set of **continuous sectors**.
- If the data blocks are packed together, the number of disk head movements will be small because the data is placed in neighboring sectors. However, if the data blocks are **scattered**, the number of disk head movements will be large. Note that the number of disk head movements determines the time in accessing the file!

The cause is the **external fragmentation**, when deleting old data and as a result leaving unallocated spaces between allocated spaces. To deal with data fragmentation better, we need **defragmentation**. In other words, we need to pack the data blocks of a file together.

Windows provides the Microsoft Drive Optimizer. Linux provides *e2defrag* for ext2/3 and *e4defrag* for ext4 file system.

Cache

A **cache** is a temporary storage designed to save time and cash. In the computing world, the **faster** the device is, the **more cache** it costs. Disks are cheaper than the RAM (memory), and the RAM is cheaper than the CPU registers.

For instance, you expect a particular file to be frequently accessed (*e.g.*, the root directory file). To save disk access time, you want to keep it in the memory at all times. That piece of memory is called a **cache**.

However, you do not have a lot of cash to buy a large RAM. Sometimes, when the cache space is running out, the root directory file has to be written back to the disk. The **cache replacement algorithm** determines this, and is controlled by the kernel.

Buffer cache

The **buffer cache** (block cache) includes all the things related to the file system that can be read from the disk, for instance, inode entries, (partial) FAT table, directory entries, and data blocks.
- The data stays in the buffer cache even after the corresponding file is closed! For example, in Lab 3, we test our encoding algorithm using very large input files, and we may want to do it multiple times. If we do not clear it from the buffer cache immediately, then we can read from the memory except for the first time, which is much faster than reading from the disk every single time.

When a disk access is finished (**primary copy**), that data is kept in the **kernel-space** memory for caching purpose (**secondary copy**). The data is kept in kernel space since no matter what, we need to invoke system calls to do those I/O, which happen in kernel mode. When someone wants to read the same piece of data, the secondary copy is returned without any disk access.

When someone wants to write cached data, two approach can be considered.
- **Write-through**: both the primary and the secondary copies are updated together. The OS has to wait for the disk to finish updating. (Windows uses this approach for reliability reasons.)
- **Write-back**: the secondary copy is updated first. The update of the primary copy is **delayed**. (Linux uses this approach for performance issues.)

Cache replacement algorithm

Least recently used (LRU)

Every cached item is associated with an **age**. After a cache access has happened, the age of the accessed item is set to 0, and the age of other items are incremented by 1. For performance reasons, the cached items are sorted according to the ages.

When a read request cannot find the requested item in the cache (**cache miss**), data is read from the disk. If there are enough cache space, the new item is directed written to the cache space. Otherwise, the **least recently used** (oldest) **item** will be written back to the disk.

File system consistency and recovery

When everything is normal, to create a file, the kernel does the following:

1. Read free space management data from the disk.
2. Update free space management data in the disk.
3. Read file allocation information from the disk.
4. Update file allocation information and write it to the disk.
5. Read directory entry from the disk.
6. Update directory entry in the disk.
7. Write data to allocated blocks.

If a crash happens somewhere:

- After Step 1: no problem.
- After Step 2: free space is allocated to nothing -> the FS loses some free space.
- After Step 3: same as the previous case.
- After Step 4: allocated spaces are chained together now, but the space is not associated to any files -> the FS again loses some free space.
- After Step 5: same as the previous case.
- After Step 6: the directory is pointing to uninitialized data, so the content of the file is wrong.

We need to detect those inconsistencies after the OS comes back, and recover from them. There are some file system checking and recovery **tools**: *fsck* (Linux/macOS), *chkdsk* and *scandisk* (Windows).

Error if crashing after Step 2/3

Checking: Check the free space management data against the entire set of allocated blocks.
fsck can construct a **bitmap** of allocated blocks using the inode table. Then it compares this newly-generated bitmap with the bitmap stored on the disk.

Recovering:
fsck can discover missing blocks and then reset the **free space bitmap**, and reset the fields in the superblock (if necessary).

Error if crashing after Step 4/5

Checking: Check the entire directory (entry) tree, and locate the allocated inodes by referring to the inode number of every directory entry. Then, construct the **block allocation bitmap** based on the discovered inodes, and compare with the **free space bitmap** as well as the **inode allocation tables** on the disk. *fsck* can discover missing blocks and **dangling inodes**, *i.e.*, the inodes created without any directory entries referring to.

Recovering:
fsck can reset the **free space bitmap**, reset the fields in the superblock (if necessary), and delete the **dangling inodes** (similar to the garbage collection process).

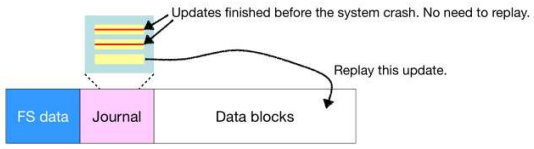
Error if crashing after 6/7

Checking: Journaling file systems. A **journal** is a log book for the file system, which is kept **inside** the file system, *i.e.*, on the disk. In order to make use of the journal, a set of file system becomes an **atomic operation** (either all operations are completed successfully or no operation is completed). A **transaction** marks all the changes that will be done to the file system, and every transaction is written to the journal.

All the changes to the file system are written to the journal first, including changes in the **metadata** and the **file content**. Then, the FS-related system call returns to the user process. Meanwhile, the entries in the journal are **replayed** and the changes are reflected to the actual file system.

The journal stores the update actions. The kernel updates the actual file system according to the actions written in the journal. When a change is **committed** to the disk, they will be marked as done. When all changes are committed, the transaction in the journal will be removed.

Recovering:



Issue: what if a crash happens while the system is writing the journal?

During recovery, we will find that some action in a transaction is incomplete, leading to an incomplete operation. An incomplete operation implies an incomplete transaction. Due to the **atomocity property**, the entire transaction has to be rolled back, *i.e.*, the changes that have been performed according to the transaction should also be undo.

Speed vs reliability

Data is written twice: once in the journal and once in the actual file system. Here are some different **journaling modes** in ext4:

- **Journal mode**: slowest, but most reliable. We write metadata and the file content to the journal first, and then to the file system.
- **Ordered mode**: default, pretty good speed and reliability. The file content is forced to be written to the file system first, then we write the metadata to the journal.
- **Write-back mode**: fastest, but incorrect data may appear. Only metadata is written to the journal.

RAID

- We know that disks are super slow compared with CPU, and disk speed improves slowly compared with CPU.
- Also, in single-disk systems, any disk failure leads to data loss.
- Moreover, a single fast, reliable disk (*e.g.*, for a data center) is extremely expensive.

RAID (Redundant Array of Inexpensive/Independent Disks) uses redundancy to improve both performance and reliability.
- Use **redundant array of inexpensive disks** as one storage unit.
- Fast: **simultaneously** read and write disks in the array.
- Reliable: use **parity** (*i.e.*, XOR) to detect and correct errors.

RAID has seven different RAID levels (0--6), achieving different performance and reliability.

RAID 0: non-redundant striping

Data are striped across all disks in an array. There is no parity.
- Good performance: with N disks, we can achieve roughly N times speedup.
- Poor reliability: any disk failure leads to data loss.

RAID 1: mirroring

Keep a **mirrored** (shadow) copy of data.
- Good read performance: roughly 2 times speedup.
- Good reliability: it can recover from one disk failure.
- But high cost: each data disk requires a <u>parity disk</u>.

RAID 4: striping with parity

A set of data blocks (**parity group**) is striped across data disks. The parity disk is the XOR of all other disks. For a single bit, if there is an odd number of 1's, then the parity disk will show 1, and otherwise the parity disk will show 0. <u>So there is an invariant: for each row (the corresponding bit of all disks including the parity disk), there is an even number of 1's.</u>
- Good read performance: roughly N-1 times speedup (but not for random writes)
- Good reliability: it can recover from one disk failure using the XOR.
- But random write performance is poor: every write needs to write data to the parity disk, so heavy load will be posed on the parity disk.

RAID 5: striping with distributed parity

Parity blocks are **distributed** across all disks in a round-robin fashion.
- No too heavy load: there is no single parity disk
- Good performance: even for random writes
- Good reliability: it can recover from one disk failure using the XOR.
- But vulnerable during recovery: we need to read from all other disks during recovery, so they will have much load, and if a second failure happens during this process, this will lead to a total loss.

RAID 6: striping with double distributed parity

Similar to RAID 5, except that there are **two parity** blocks per parity group. Note that we use some parity method (independent of XOR) to compute Q.
- Same advantage as RAID 5.
- Better reliability: it can recover from two disk failures.

Summary

We can build **larger, faster, and more reliable** storage out of inexpensive disks.